CONRAD CHALLENGE 2025–26

TECHNICAL PROJECT FILE

HydroScan

A low-cost device for optical detection of microplastic and particulate contamination in water samples

Microplastics
HIGH 74% n74

Project team

Team member	Responsibility
Anay Mehta	Electronics and hardware
Yuvraj Nikum	Detection software

Project links

Source code **github.com/AnayMehta976/HydroScan**
Website **hydro-scan-six.vercel.app**

Contents

01 Problem statement 3
02 Background 4
03 The device 6
04 Hardware 7
05 Construction 8
06 Edge computing: running the model on the device 10
07 Data collection 11
08 The detection model 12
09 Experiments 14
10 Validation and its limits 16
11 Known limitations 17
12 Cost 18
13 Future work 19
14 My contribution (Anay Mehta) 20
15 Bibliography 21

Figures

1 Optical layout of the imaging chamber 5
2 Assembled device, front view 6
3 Internal assembly 8

Tables

1 Comparison of methods 4
2 Hardware components 7
3 Dataset summary 11
4 Image-processing pipeline 12
5 Feature set 13
6 Class-to-level mapping 13
7 Results on the held-out test set 14
8 Cost breakdown 18

1 Problem statement

Microplastics and fine particulates are present throughout drinking water sources. They cannot be seen by the people drinking the water.

Laboratory methods can identify them accurately. FTIR and Raman spectroscopy confirm a particle's polymer chemically, and microscopy gives particle counts. These methods are expensive and slow. They need trained operators and fixed facilities. In practice the choice is a full laboratory analysis or no testing at all, and most water is never screened.

HydroScan targets that gap. It is a cheap, repeatable, on-site first-level screen that flags whether a sample deserves proper testing. It is not a lab replacement.

2 Background

2.1 Why microplastics matter

Microplastics are small fragments of plastic. Some are manufactured at that size. Others form when larger plastic items break down, or are shed from synthetic textiles and from the wear of plastic products. They do not readily degrade, so once they enter a water source they stay there. Their effects on human health are still being studied [1].

Microplastics also work as a marker of contamination in general. Water polluted by waste water tends to carry a high load of particulates, plastic and otherwise. A sample with many suspended particles is more likely to have been exposed to a source of contamination, whatever those particles turn out to be.

HydroScan relies on this. A high particle reading does not identify the contaminant. It is a reason to send the sample for proper testing.

2.2 Existing methods

Table 1 compares HydroScan with the established ways of examining particles in water.

Table 1. Comparison of methods.

	FTIR / Raman	Microscopy	Turbidity meter	HydroScan
What it measures	Vibrational spectrum of each particle, which identifies its polymer	Count, size and shape of particles under magnification	Bulk cloudiness from light scattered by suspended matter	Count, size, shape, colour and brightness of light-scattering particles
Confirms plastic chemically	Yes	No	No	No
Operator skill	High; trained analyst	Moderate to high	Low	Low; one button
Setting	Fixed laboratory	Laboratory	Field or laboratory	On site
Speed	Slow; needs sample preparation	Slow; counting is labour-intensive	Fast	Fast; three frames per test
Relative cost	High	Moderate to high	Low to moderate	Low; no cost per test
Output	Polymer identity of each particle	Particle counts and sizes	Single turbidity value	LOW / MEDIUM / HIGH, a relative index and a particle count

Only FTIR and Raman spectroscopy confirm chemically that a particle is plastic. Microscopy and HydroScan both rely on how particles look. A turbidity meter is the closest match to HydroScan in what it physically measures, since both respond to light scattered by suspended matter. A turbidity meter reports one bulk value for the whole sample. HydroScan counts and sizes individual particles and reports a category.

2.3 Side illumination and light scattering

A low-cost camera module has limited resolution and sensitivity. Under ordinary lighting from above, a small, nearly transparent fragment changes the image very little and is easy to miss.

HydroScan changes what reaches the camera by lighting the sample from the sides instead. A single white LED strip runs continuously along three of the inside walls of a sealed chamber, level with the glass dish, and shines across the sample while the camera looks straight down. Side light that passes cleanly through the water travels across the field of view rather than into the lens. Light reaches the camera mainly when a particle scatters it upward.

Particles therefore appear as bright specks against a darker background. This is the same principle as dark-field imaging. In the current prototype the inside of the chamber is lined white rather than matte black, so the background is not fully dark: the walls reflect some LED light and the illumination across the dish is uneven. The software compensates for this with local contrast equalisation (CLAHE), a white top-hat filter and a glare mask (Section 8.1). A matte-black interior, which would give true dark-field contrast, is listed under future work. Figure 1 shows the layout used in HydroScan.

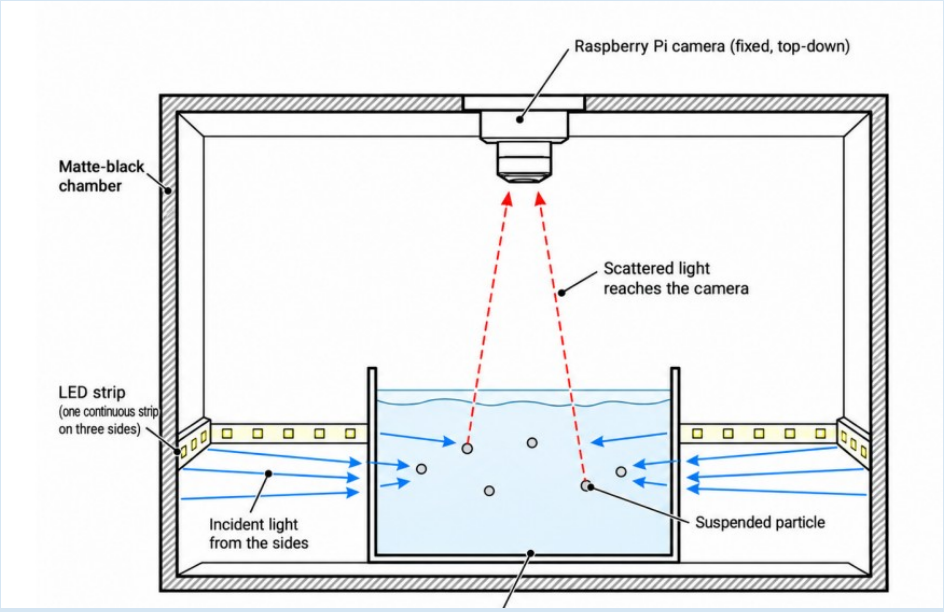


Figure 1. Optical layout of the imaging chamber. One continuous LED strip on three inside walls lights the dish from the sides (blue); suspended particles scatter light up to the fixed top-down camera (red).

3 The device

A water sample is poured into a clear glass dish. The dish sits on a base that holds it at a fixed height inside a sealed chamber. The chamber is black outside and white inside, and blocks ambient light. A single white LED strip runs along three of the inside walls and lights the sample from the sides. A Raspberry Pi Camera Module v3 NOIR, mounted at the top of the chamber, images the sample from directly above.

To run a test, the user presses the scan button. The LEDs switch on, the Raspberry Pi captures three frames, extracts numerical features from them, classifies the sample and shows the result on a 16×2 LCD: the word “Microplastics” on the first line, and the level (LOW, MEDIUM or HIGH), a relative index and the particle count on the second, for example “HIGH 74% n74”.

The device works fully offline and runs from a USB-C power bank. It needs no reagents, and there is no cost per test.



Figure 2. Assembled device, front view.

4 Hardware

Table 2. Hardware components.

Component	Purpose
Raspberry Pi 4	Runs the full pipeline on the device: LED control, image capture, feature extraction and classifier inference, with no internet connection. Chosen so the unit works offline anywhere.
Camera Module v3 NOIR	Connected to the CSI port. Images the sample from directly above at 1280×720.
White LED strip (one continuous strip on three inside walls)	Side lighting so that particles scatter light toward the camera. Switched from GPIO27 through a MOSFET, so it is on only during a scan.
16×2 I2C LCD	Shows the level, relative index and particle count. Connected over I2C (SDA GPIO2, SCL GPIO3), address 0x27.
Momentary push button	Starts a test. Wired from GPIO17 to ground, using the Pi's internal pull-up.
Glass sample dish	Holds the sample; clear walls let the side light in.
Enclosure	Sealed chamber, black outside and white inside. Blocks ambient light; the base keeps the dish at a fixed height.
Power components	USB-C power bank and cabling. Power the whole unit from one supply.
Miscellaneous	Jumper wires, MOSFET, resistors, mounting hardware and adhesive.

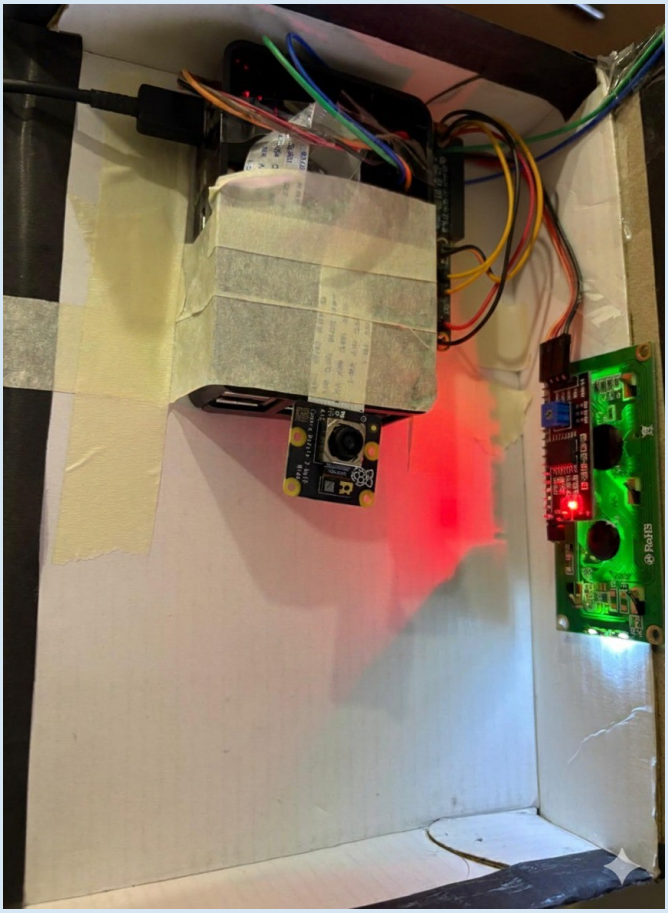


Figure 3. Internal assembly showing the Raspberry Pi, camera, LED wiring and LCD.

5 Construction

5.1 Optical chamber

The chamber is sealed so that the only light inside comes from the LEDs. Ambient light varies from room to room and hour to hour; letting it in would change the image of the same sample from one test to the next.

The outside of the enclosure is black and the inside is white. The white lining spreads the LED light more evenly around the dish, but it also reflects light back into the camera's view and raises the brightness of the background. The light scattered by a small particle is faint, and a brighter background makes it harder to pick out. In this prototype that trade-off is handled in software (Section 8.1). Lining the interior matte black would reduce stray reflections at the source and is listed under future work.

The LED strip is mounted at the level of the dish rather than above it, so its light travels across the sample and the camera receives mainly the light that particles scatter upward, rather than light reflected straight off the water surface or the bottom of the dish.

The strip runs continuously along three walls rather than one. With a single light source, particles on the far side of the dish would be lit more weakly than those near the LEDs, and particles could

shadow one another. Lighting from three sides keeps a particle lit wherever it sits in the dish. This reduces shadowing and reduces bias from particle position. The fourth side is left unlit, which still leaves that edge of the dish slightly dimmer; CLAHE evens out the remaining gradient in software.

5.2 Camera mount

The camera is rigidly fixed above the sample, pointing straight down, and the base holds the dish at a fixed height. This matters because of what the classifier measures. Its features include particle count and particle area in pixels. If the camera moved closer, the same particles would cover more pixels. If it tilted, the visible region and the apparent particle shapes would change. In either case the same sample would produce different numbers. The fixed mount keeps the camera's distance and angle the same for every test.

5.3 Circuitry

The LCD connects to the Raspberry Pi over I2C on GPIO2 and GPIO3. The push button is wired from GPIO17 to ground and uses the Pi's internal pull-up resistor. The LED strip is switched from GPIO27 through a MOSFET, so the software turns it on for a scan and off afterwards. The camera connects to the Pi's CSI port. A single USB-C power bank powers the whole unit.

5.4 Operating sequence

1. Place the glass dish on the base in the chamber.
2. Pour the water sample into the dish.
3. Let the sample settle.
4. Close the lid.
5. Press the scan button.
6. The Pi switches the LEDs on and captures three frames.
7. Each frame passes a quality check for focus and brightness; frames that fail are discarded.
8. The Pi processes the accepted frames, takes the median particle count, extracts the features and classifies the sample.
9. The LCD shows "Microplastics" and the level, relative index and count, for example "HIGH 74% n74".
10. An annotated image and a log entry are saved on the device, and the LEDs switch off.

5.5 Purpose of the hardware design

The hardware exists to hold imaging conditions constant. Lighting, background, camera distance and camera angle are the same for every sample. This is what makes the model's inputs meaningful. When two samples produce different numbers, the difference comes from the samples.

6 Edge computing: running the model on the device

Every part of the detection pipeline runs on the Raspberry Pi inside the box. The image is captured, processed and classified on the device, and the result goes straight to the LCD. Nothing leaves the unit at any point.

The obvious alternative would have been to send the image to a server and get a classification back. That would have made the hardware cheaper and simpler, and it would also have made the device useless in most of the places it is meant for. A water screening tool that needs a working internet connection cannot be used in a remote village, in a disaster zone, or anywhere the network is down, and those are exactly the conditions where nobody knows what is in the water. Choosing to process locally meant accepting a more expensive processor so the device would work with no connectivity at all.

This constrained the hardware. The processor had to be able to run an OpenCV pipeline and classifier inference within seconds, from a USB power bank, with no GPU and no active cooling. The Raspberry Pi 4 was chosen on that basis: enough compute to run inference locally, a camera interface, GPIO for the button and LEDs, I2C for the display, and a 5 V supply that a standard power bank can provide.

The model is a Random Forest of 200 trees, which fits these constraints well. The trained model is stored on the device as a joblib file and loaded when a scan runs, so there is no download step and no setup. Inference on 11 numerical features is cheap, and there is no GPU requirement. A neural network running on raw images would have needed more memory, more power and more time per test, and is unlikely to help on a dataset this small.

Three consequences of processing at the edge:

- The device works with no internet, no mobile signal and no mains power.
- Nothing about the user’s water leaves the device, so no data is sent anywhere.
- There is no per-test server cost, which is part of why the cost per test is nil.

7 Data collection

No certified microplastic standards were available, and building a large set of physically prepared and independently measured samples was beyond the scope of the project. The training dataset is therefore synthetic. It was generated in software by the script scripts/generate_ml_dataset.py, which draws images that imitate what the HydroScan camera sees: bright particles on a lit background inside the chamber.

Each image is drawn at the camera's resolution of 1280×720. The number of plastic-like particles in each image is chosen at random from a range that defines its class:

- **clean:** 0 to 5 particles
- **low:** 6 to 19 particles
- **medium:** 22 to 55 particles
- **high:** 65 to 130 particles
- **non_plastic:** 8 to 30 particles of organic or sandy debris, which are darker, more coloured and less regular than plastic

The gaps between the class ranges keep each class distinct. To make the images closer to real captures, the generator adds an uneven LED illumination gradient and hotspot, specular glare spots, air bubbles, some cross-contamination (a few debris particles in plastic samples and a few plastic-like particles in debris samples), variation in background tint, and random blur.

The labels are known by construction. They record how many particles the generator drew in each image. They are not a measured concentration, and no image in the dataset is a photograph of a real water sample.

Table 3. Dataset summary.

Parameter	Value
Source	Synthetic images (scripts/generate_ml_dataset.py)
Image size	1280×720
Total images	500
clean	100
low	100
medium	100
high	100
non_plastic	100

8 The detection model

The detection software described in this section was designed and built by Yuvraj Nikum.

The model never sees raw pixels. An OpenCV pipeline [5] first reduces each captured frame to 11 numerical features. The classifier works on those features alone.

8.1 Image-processing pipeline

Table 4. Image-processing pipeline, applied in order to every captured frame.

Step	Operation	Purpose
1	Quality check	Reject frames that are out of focus (Laplacian variance below 50) or too dark or too bright (mean brightness outside 18–230).
2	Crop	Restrict analysis to the central 80% of the frame, where the water is.
3	CLAHE	Even out the uneven LED lighting (clip limit 2.0, 8×8 tiles) [3].
4	Glare mask	Mask saturated LED glare (brightness 245 and above, areas of 60 pixels or more) so it is not counted.
5	Gaussian blur	Suppress sensor noise.
6	White top-hat (Otsu) and adaptive threshold	Isolate bright specks from the background. A pixel must pass both to count.
7	Morphological opening	Remove single-pixel noise.
8	Watershed	Split particles that touch [4].
9	Particle scoring	Score each particle as plastic-like or debris from its colour and shape.

The particle score combines three terms: low colour saturation, high brightness and a regular shape. In the software it is 0.40 × (1 − saturation/180) + 0.30 × ((brightness − 60)/160) + 0.30 × shape, where the shape term is 0.5 × solidity + 0.3 × circularity + 0.2/elongation. A particle scoring at least 0.40 is counted as plastic-like; the rest, and any particle with saturation of 90 or more, are counted as debris.

Three frames are captured per test. The final particle count is the median across the frames that passed the quality check, which removes the effect of a single bad frame, and the other features are averaged across those frames. The software also saves an annotated copy of the frame closest to the median, with plastic-like particles outlined in green and debris in orange, and appends the result to a log file.

8.2 Feature set

Table 5. The 11 features computed for each frame, by their names in the software, grouped here by what they describe.

Group	Features
Count	count, debris_count, density_per_megapixel
Size	mean_area, size_small, size_medium, size_large
Shape	mean_circularity, mean_solidity
Appearance	mean_plastic_score, mean_brightness

8.3 Classifier

The classifier is a Random Forest [2] of 200 trees, built with scikit-learn [6]. The trained model is saved to a file and loaded on the Raspberry Pi with joblib.

A Random Forest was chosen over a neural network because of the shape of the input. The model receives a small set of engineered features. On that kind of input a Random Forest trains reliably from a modest dataset, and it runs fast on the Pi without a GPU. The confidence reported with each result is the forest's highest class probability.

8.4 Output mapping

The model predicts one of five classes. These are mapped to the three levels shown on the LCD.

Table 6. Class-to-level mapping.

Predicted class	Displayed level
clean	LOW
low	LOW
medium	MEDIUM
high	HIGH
non_plastic	LOW, with the note "Mostly non-plastic debris (sand/organic)"

The LCD also shows a relative index, the plastic-like particle count divided by 100 and capped at 100%. This is a fixed scale for comparing samples, not a concentration.

8.5 Fallback

If the model file or its libraries are missing, or the prediction fails, the software falls back to threshold rules on the particle count: fewer than 20 particles is LOW, fewer than 60 is MEDIUM, and 60 or more is HIGH. The fallback runs only in those cases. In normal operation every result comes from the Random Forest.

9 Experiments

Experiment 1 used the synthetic dataset described in Section 7. Experiment 2 used the physical prototype.

Experiment 1. Classification across known levels

Aim. To measure how accurately the model classifies images whose particle level is known.

Method. The 500 images were split at random into a training set (75%, 375 images) and a held-out test set (25%, 125 images), stratified so each class is equally represented (scikit-learn train_test_split, random_state 42). The model never trained on the test images. Accuracy was measured on the held-out set. Separately, the same features were evaluated with 5-fold stratified cross-validation.

Result. Accuracy on the held-out test set was 88.0% (110 of 125 images). Accuracy under 5-fold cross-validation was 92.2% ± 2.3%. Every high image was classified correctly. Most errors were between the clean and low classes: recall for low was 0.64. Per-class figures are in Table 7.

Conclusion. Within this synthetic setup, the model separates the particle levels well. Its weakest boundary is between clean and low, the two classes closest in particle count. Both classes display as LOW, so this confusion does not change the level shown to the user. Because train and test images come from the same generator, these figures do not show performance on real water (Section 10).

Table 7. Results on the held-out test set (25 images per class).

Class	Precision	Recall
clean	0.73	0.88
low	0.80	0.64
medium	0.92	0.96
high	1.00	1.00
non_plastic	0.96	0.92
Total (accuracy)		**0.88**

Experiment 2. Prototype testing

Aim. To check that the physical device works as a system and gives distinguishable images for different sample levels.

Method. A working prototype was built and run on water samples across the low, medium and high ranges. The scan cycle was repeated many times. The device was also handed to peers and mentors to operate.

Result. Samples across the three ranges produced visibly distinct scattering patterns. Dozens of repeated scan cycles ran with no consumables. Peers and mentors ran tests without training.

Conclusion. The prototype works end to end and is simple to operate. These checks were qualitative. The number of trials and the spread of readings for a repeated sample were not recorded, so this experiment does not measure repeatability numerically. A controlled repeatability test is listed under future work.

10 Validation and its limits

HydroScan has not been validated against an independent analytical reference. No sample was run through both HydroScan and an FTIR, Raman or microscopy analysis.

The model has been trained and tested only on synthetic images. The 88.0% and 92.2% figures in Section 9 show that it separates the classes the generator produces. The test images were held out from training, but they come from the same generator with the same particle shapes, colours and lighting effects. The figures do not show how the device performs on photographs of real water, or that a given reading matches a given amount of microplastic in a real sample.

Real validation would mean two things: retraining the model on labelled photographs taken inside the device, and running identical samples through HydroScan and a reference laboratory method and comparing the results. Neither has been done yet.

11 Known limitations

No.	Limitation	Description
1	**Optical measurement only**	HydroScan infers that a particle is plastic-like from its brightness, colour, size and shape. It cannot chemically confirm that a particle is plastic. A bright, compact mineral fragment can read as plastic-like.
2	**Responds to all particles**	Light scattering happens at every particle, plastic or not. Physically, the device is closer to a turbidity screen than to a microplastic-specific assay.
3	**Synthetic training data**	The model was trained and tested on generated images, not on photographs of real samples. Real particles, real water and the real chamber will differ from the generator in ways the model has not seen.
4	**Light-coloured debris**	The plastic score separates debris mainly by colour. Light-tan organic particles can score above the 0.40 threshold: in a check on synthetic organic particles, about 40% were counted as plastic-like.
5	**Resolution**	The camera images the whole dish at 1280×720, so each pixel covers a sizeable patch of the sample. Very small microplastics fall below what the device can resolve and are not counted.
6	**Background brightness**	The white interior raises background brightness and reflections. Software compensates, but faint particles near bright areas can still be missed.
7	**Categorical output**	The output is one of three categories. The percentage on the LCD is a relative index (particle count on a fixed scale of 100), not a concentration.

12 Cost

Table 8. Cost breakdown. Prices in Indian rupees (₹).

Item	Cost (₹)
Raspberry Pi 4	4,000
Camera Module v3 NOIR	2,500
Miscellaneous (wires, MOSFET, resistors, mounting)	450
Power components	300
Enclosure	300
16×2 I2C LCD	250
White LEDs	200
Total	**≈ 8,000 (about US$89)**
Cost per test	Nil

Once the device is built, a test costs nothing: there are no reagents or consumables.

The Raspberry Pi and the camera module together make up ₹6,500, about 81% of the cost. Reducing the unit cost therefore means cheaper compute and imaging, for example a Raspberry Pi Zero 2 W. Economising on the small parts would make little difference.

13 Future work

1. Collect real labelled samples by photographing known samples inside the device, and retrain the model on them.
2. Validate against a reference laboratory method by running identical samples through HydroScan and FTIR, Raman or microscopy.
3. Line the chamber interior matte black for true dark-field contrast.
4. Run a controlled repeatability test: scan the same sample many times and record the spread of counts and levels.
5. Run controlled-variation tests to quantify how much readings drift with lighting and camera position.
6. Improve the plastic score so that light-coloured organic debris is not counted as plastic.
7. Add a fluorescence stage for genuine chemical discrimination.
8. Move from three categories to an estimated concentration.
9. Reduce hardware cost with cheaper compute and imaging.

14 My contribution (Anay Mehta)

I designed and built the electronics and physical device: the sealed optical chamber and its three-sided LED lighting, the fixed top-down camera mount and the base that holds the dish at a fixed height, and the wiring and integration of the Pi, camera, LEDs, LCD and button into one unit running from a single USB-C power bank. That included switching the LED strip from a GPIO pin through a MOSFET, so the lights come on only for a scan.

Choosing a processor that could run the classifier locally, and building the device around that constraint, was part of the hardware design. It decided the power budget, the thermal behaviour and the cost of the unit. Yuvraj's model had to fit on the board I picked, and the board had to be able to carry it.

My focus was making it operate as one reliable system and making sure the hardware holds imaging conditions constant enough for the measurements to mean anything.

The OpenCV pipeline and Random Forest classifier were led by Yuvraj Nikum. I understand how they consume the images my hardware produces, but the design decisions within that software are his.

15 Bibliography

[1] World Health Organization (2019). Microplastics in drinking-water. Geneva: WHO.

[2] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[3] Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In P. Heckbert (Ed.), Graphics Gems IV (pp. 474–485). Academic Press.

[4] Vincent, L., & Soille, P. (1991). Watersheds in digital spaces: an efficient algorithm based on immersion simulations. IEEE Transactions on Pattern Analysis and Machine Intelligence, 13(6), 583–598.

[5] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[7] Raspberry Pi Ltd. Camera documentation (Camera Module 3). raspberrypi.com/documentation/accessories/camera.html